

For program 1

```
#include <stdio.h>

#define MAX 10

/* Function Prototypes */
void readMatrix(int mat[MAX][MAX], int rows, int cols);
void addMatrix(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols);
void subtractMatrix(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols);
void multiplyMatrix(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int r1, int c1, int c2);
void transposeMatrix(int mat[MAX][MAX], int trans[MAX][MAX], int rows, int cols);
void displayMatrix(int mat[MAX][MAX], int rows, int cols);

int main()
{
    int A[MAX][MAX], B[MAX][MAX], C[MAX][MAX], T[MAX][MAX];
    int r1, c1, r2, c2;
    int choice;
    int matrixRead = 0;

    while (1)
    {
        printf("\n===== MATRIX MENU =====\n");
        printf("1. Read Matrices\n");
        printf("2. Addition\n");
        printf("3. Subtraction\n");
        printf("4. Multiplication\n");
        printf("5. Transpose (Matrix A)\n");
        printf("6. Display Matrices\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");

        /* Parameter Safety: Validate input */
        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input!\n");
            while(getchar() != '\n');
            continue;
        }

        /* Parameter Safety: Validate menu choice */
        if (choice < 1 || choice > 7)
        {
            printf("Invalid menu choice!\n");
            continue;
        }

        switch(choice)
        {
            case 1:

                printf("Enter rows and columns of Matrix A: ");
```

```

if(scanf("%d%d",&r1,&c1)!=2)
{
    printf("Invalid input!\n");
    return 0;
}

if(r1<=0 || c1<=0 || r1>MAX || c1>MAX)
{
    printf("Invalid dimensions for Matrix A!\n");
    break;
}

printf("Enter rows and columns of Matrix B: ");

if(scanf("%d%d",&r2,&c2)!=2)
{
    printf("Invalid input!\n");
    return 0;
}

if(r2<=0 || c2<=0 || r2>MAX || c2>MAX)
{
    printf("Invalid dimensions for Matrix B!\n");
    break;
}

printf("\nEnter Matrix A:\n");
readMatrix(A,r1,c1);

printf("\nEnter Matrix B:\n");
readMatrix(B,r2,c2);

matrixRead=1;
break;

```

case 2:

```

if(matrixRead==0)
{
    printf("Read matrices first!\n");
    break;
}

if(r1!=r2 || c1!=c2)
{
    printf("Addition not possible.\n");
    break;
}

addMatrix(A,B,C,r1,c1);

printf("\nResult of Addition:\n");
displayMatrix(C,r1,c1);

break;

```

case 3:

```
if(matrixRead==0)
{
    printf("Read matrices first!\n");
    break;
}

if(r1!=r2 || c1!=c2)
{
    printf("Subtraction not possible.\n");
    break;
}

subtractMatrix(A,B,C,r1,c1);

printf("\nResult of Subtraction:\n");
displayMatrix(C,r1,c1);

break;
```

case 4:

```
if(matrixRead==0)
{
    printf("Read matrices first!\n");
    break;
}

if(c1!=r2)
{
    printf("Multiplication not possible.\n");
    break;
}

multiplyMatrix(A,B,C,r1,c1,c2);

printf("\nResult of Multiplication:\n");
displayMatrix(C,r1,c2);

break;
```

case 5:

```
if(matrixRead==0)
{
    printf("Read Matrix A first!\n");
    break;
}

transposeMatrix(A,T,r1,c1);

printf("\nTranspose of Matrix A:\n");
displayMatrix(T,c1,r1);
```

```

        break;

    case 6:

        if(matrixRead==0)
        {
            printf("Read matrices first!\n");
            break;
        }

        printf("\nMatrix A:\n");
        displayMatrix(A,r1,c1);

        printf("\nMatrix B:\n");
        displayMatrix(B,r2,c2);

        break;

    case 7:

        printf("Program Terminated.\n");
        return 0;
    }
}

return 0;
}

/* Function to Read Matrix */
void readMatrix(int mat[MAX][MAX], int rows, int cols)
{
    int i,j;

    for(i=0;i<rows;i++)
    {
        for(j=0;j<cols;j++)
        {
            scanf("%d",&mat[i][j]);
        }
    }
}

/* Function to Add Matrices */
void addMatrix(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols)
{
    int i,j;

    for(i=0;i<rows;i++)
    {
        for(j=0;j<cols;j++)
        {
            result[i][j]=a[i][j]+b[i][j];
        }
    }
}

```

```
}
```

```
/* Function to Subtract Matrices */
```

```
void subtractMatrix(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int rows, int cols)
```

```
{
```

```
    int i,j;
```

```
    for(i=0;i<rows;i++)
```

```
    {
```

```
        for(j=0;j<cols;j++)
```

```
        {
```

```
            result[i][j]=a[i][j]-b[i][j];
```

```
        }
```

```
    }
```

```
}
```

```
/* Function to Multiply Matrices */
```

```
void multiplyMatrix(int a[MAX][MAX], int b[MAX][MAX], int result[MAX][MAX], int r1, int c1, int c2)
```

```
{
```

```
    int i,j,k;
```

```
    for(i=0;i<r1;i++)
```

```
    {
```

```
        for(j=0;j<c2;j++)
```

```
        {
```

```
            result[i][j]=0;
```

```
            for(k=0;k<c1;k++)
```

```
            {
```

```
                result[i][j]+=a[i][k]*b[k][j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
/* Function to Find Transpose */
```

```
void transposeMatrix(int mat[MAX][MAX], int trans[MAX][MAX], int rows, int cols)
```

```
{
```

```
    int i,j;
```

```
    for(i=0;i<rows;i++)
```

```
    {
```

```
        for(j=0;j<cols;j++)
```

```
        {
```

```
            trans[j][i]=mat[i][j];
```

```
        }
```

```
    }
```

```
}
```

```
/* Function to Display Matrix */
```

```
void displayMatrix(int mat[MAX][MAX], int rows, int cols)
```

```
{
```

```
    int i,j;
```

```
for(i=0;i<rows;i++)
{
    for(j=0;j<cols;j++)
    {
        printf("%4d",mat[i][j]);
    }
    printf("\n");
}
```

Program-2

Program : Bank Account Management System
Concept : Parameter Passing and Parameter Safety

```
#include<stdio.h>
#include<string.h>

#define MAX_NAME 30

/* Function Prototypes */

float createAccount(int accNo, char name[], float balance);
float deposit(float balance, float amount);
float withdraw(float balance, float amount);
float checkBalance(float balance);
void displayAccount(int accNo, char name[], float balance);

int main()
{
    int choice;
    int accountNo=0;
    char customerName[MAX_NAME];
    float balance=0;
    float amount;

    int accountCreated=0;

    while(1)
    {
        printf("\n=====");
        printf("\n BANK ACCOUNT MANAGEMENT SYSTEM");
        printf("\n=====");
        printf("\n1. Create Account");
        printf("\n2. Deposit Money");
        printf("\n3. Withdraw Money");
        printf("\n4. Check Balance");
        printf("\n5. Display Account Details");
        printf("\n6. Exit");

        printf("\n\nEnter your choice : ");

        /* Parameter Safety */
        if(scanf("%d",&choice)!=1)
        {
            printf("\nInvalid input!");
            while(getchar()!='\n');
            continue;
        }

        if(choice<1 || choice>6)
```

```

{
    printf("\nInvalid Menu Choice.");
    continue;
}

switch(choice)
{

case 1:

    printf("\nEnter Account Number : ");

    if(scanf("%d",&accountNo)!=1)
    {
        printf("\nInvalid Account Number.");
        while(getchar()!='\n');
        break;
    }

    if(accountNo<=0)
    {
        printf("\nAccount Number must be positive.");
        break;
    }

    while(getchar()!='\n');

    printf("Enter Customer Name : ");
    fgets(customerName,MAX_NAME,stdin);

    customerName[strcspn(customerName,"\n")]='\0';

    if(strlen(customerName)==0)
    {
        printf("\nCustomer Name cannot be empty.");
        break;
    }

    printf("Enter Initial Balance : ");

    if(scanf("%f",&balance)!=1)
    {
        printf("\nInvalid Balance.");
        while(getchar()!='\n');
        break;
    }

    if(balance<0)
    {
        printf("\nInitial Balance cannot be negative.");
    }

```



```
        break;
    }

    balance=createAccount(accountNo,customerName,balance);

    accountCreated=1;

    printf("\nAccount Created Successfully.\n");

    break;
```

case 2:

```
if(accountCreated==0)
{
    printf("\nCreate Account First.");
    break;
}

printf("\nEnter Deposit Amount : ");

if(scanf("%f",&amount)!=1)
{
    printf("\nInvalid Amount.");
    while(getchar()!='\n');
    break;
}

if(amount<=0)
{
    printf("\nDeposit Amount must be positive.");
    break;
}

balance=deposit(balance,amount);

printf("\nDeposit Successful.");

break;
```

case 3:

```
if(accountCreated==0)
{
    printf("\nCreate Account First.");
    break;
}

printf("\nEnter Withdrawal Amount : ");
```

```

if(scanf("%f",&amount)!=1)
{
    printf("\nInvalid Amount.");
    while(getchar()!='\n');
    break;
}

if(amount<=0)
{
    printf("\nWithdrawal Amount must be positive.");
    break;
}

if(amount>balance)
{
    printf("\nInsufficient Balance.");
    break;
}

balance=withdraw(balance,amount);

printf("\nWithdrawal Successful.");

break;

```

case 4:

```

if(accountCreated==0)
{
    printf("\nCreate Account First.");
    break;
}

balance=checkBalance(balance);

printf("\nCurrent Balance : %.2f",balance);

break;

```

case 5:

```

if(accountCreated==0)
{
    printf("\nCreate Account First.");
    break;
}

displayAccount(accountNo,customerName,balance);

break;

```

```

        case 6:

            printf("\nThank You...\n");
            return 0;

        }

    }

    return 0;
}

/*-----*/

float createAccount(int accNo,char name[],float balance)
{
    /* Demonstrates parameter passing */

    printf("\nAccount Number : %d",accNo);
    printf("\nCustomer Name : %s",name);

    return balance;
}

/*-----*/

float deposit(float balance,float amount)
{
    balance=balance+amount;
    return balance;
}

/*-----*/

float withdraw(float balance,float amount)
{
    balance=balance-amount;
    return balance;
}

/*-----*/

float checkBalance(float balance)
{
    return balance;
}

/*-----*/

```

```
void displayAccount(int accNo,char name[],float balance)
{
    printf("\n");
    printf("\n----- ACCOUNT DETAILS -----");
    printf("\nAccount Number : %d",accNo);
    printf("\nCustomer Name : %s",name);
    printf("\nCurrent Balance: %.2f",balance);
    printf("\n-----");
}
```